

《心语时光_系统设计说明书》

1 引言

1.1 编写目的

示例内容

本系统设计说明书用于对项目系统的整体架构、功能模块、运行流程以及关键技术方案的详细描述，为系统后续开发、测试与维护提供统一的设计依据。

本文档通过对系统分层结构、AI核心模块、数据库交互以及业务流程等内容进行规范化设计，明确各模块之间的职责划分与协作关系，从而提高团队协作效率，降低后续开发过程中的沟通成本与实现风险。

此外，本说明书还将作为项目开发阶段的重要参考文档，为系统编码实现、功能扩展以及后期部署维护提供技术支持。

1.2 项目背景

示例内容

随着人工智能技术与大语言模型技术的快速发展，基于AI的智能问答、知识检索以及智能辅助分析系统逐渐成为当前软件系统的重要发展方向。传统系统通常只能提供固定规则下的信息处理能力，难以满足用户对于智能化、语义化以及个性化服务的需求。

近年来，以大语言模型（LLM）为代表的生成式人工智能技术在自然语言理解与生成领域取得了显著进展，为智能系统的构建提供了新的技术方案。与此同时，RAG（Retrieval-Augmented Generation）检索增强生成技术、Embedding语义向量技术以及Agent智能体技术的发展，使系统能够结合外部知识库实现更加准确、可靠和可扩展的智能服务。

基于上述背景，本项目拟设计并实现一套基于AI服务层的智能系统，通过结合大语言模型、向量检索以及模块化AI架构，实现智能问答、知识增强、上下文记忆以及多模块协同处理等功能，从而提高系统的智能化水平与用户交互体验。

1.3 系统目标

本系统旨在构建一套具备智能语义理解与知识增强能力的AI辅助系统，通过分层化架构设计与模块化AI服务设计，实现高效、可扩展且易维护的智能化应用平台。

系统主要目标包括：

（1）实现智能问答功能

系统能够基于用户输入内容进行语义理解，并结合大语言模型生成自然、准确的回答结果。

(2) 实现知识检索增强

系统通过Embedding向量化与向量数据库检索技术，实现基于语义相似度的知识检索，从而提升回答结果的准确性与专业性。

(3) 实现上下文记忆能力

系统能够保存用户历史会话信息，并在多轮对话过程中维持上下文连续性，提高交互体验与对话逻辑一致性。

(4) 实现模块化AI架构

系统采用模块化设计思想，将AI服务拆分为Agent、Prompt、Embedding、RAG以及Memory等多个独立模块，从而提高系统的可扩展性与后续维护能力。

(5) 提高系统可扩展性与可维护性

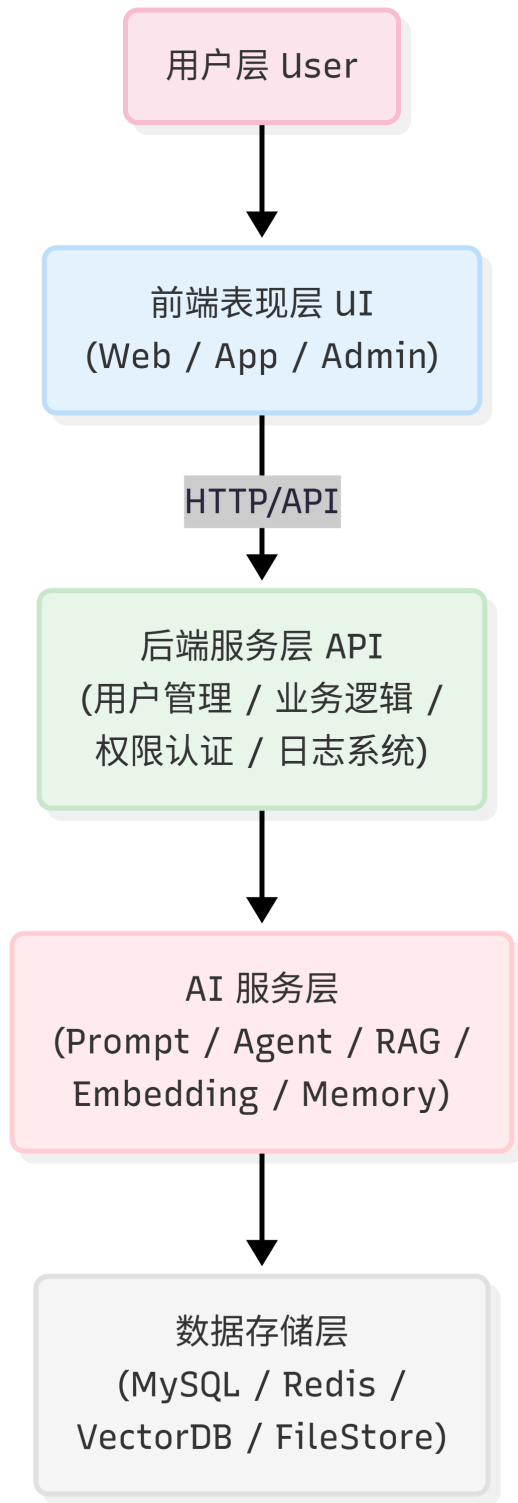
系统整体采用分层架构设计，实现前端、后端、AI服务层以及数据存储层之间的解耦，便于后续功能扩展与系统升级。

1.4 术语与缩写

缩写	含义
LLM	大语言模型
RAG	检索增强生成
API	应用程序接口

2 系统总体设计（核心）

2.1 系统架构设计



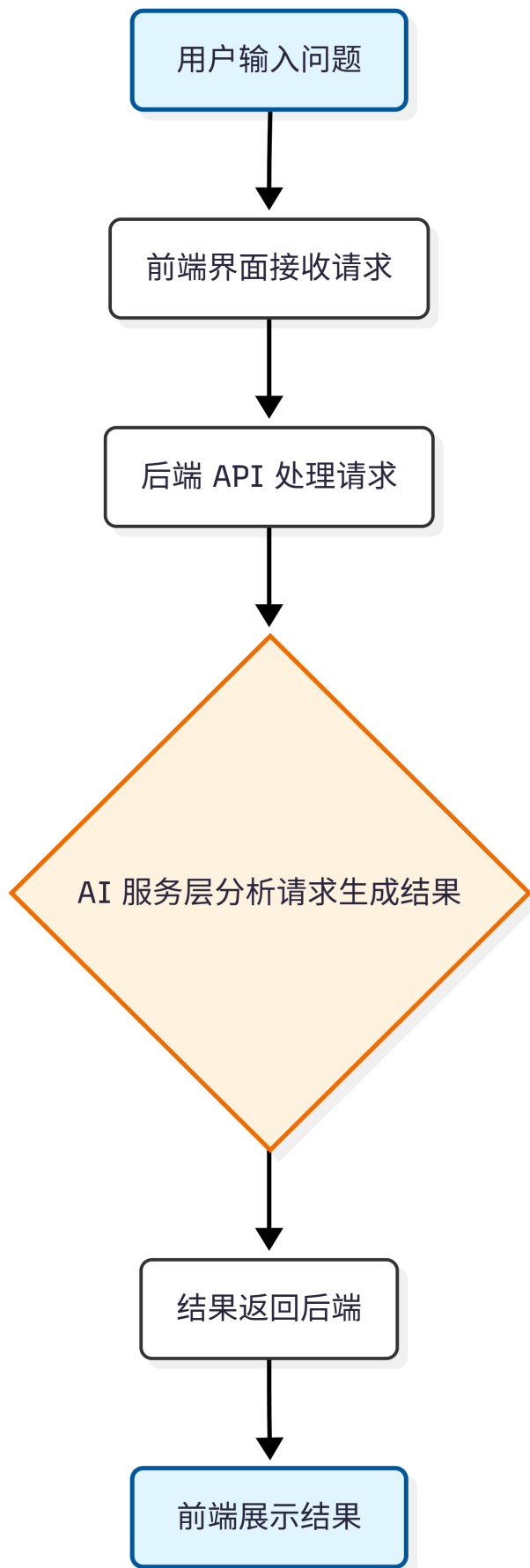
系统总体架构说明

本系统采用分层架构设计，整体由用户层、前端表现层、后端服务层、AI服务层以及数据存储层组成。

前端表现层负责用户交互与界面展示；后端服务层负责业务逻辑处理与接口管理；AI服务层负责智能问答、知识检索与模型推理；数据存储层负责用户数据、日志数据及向量数据的持久化存储。

系统各层之间通过标准化接口进行通信，从而提高系统的可扩展性与可维护性。

2.2 系统运行流程



系统运行流程说明

系统运行时，用户首先通过前端界面提交请求。前端将请求发送至后端API服务层，由后端完成参数校验与业务逻辑处理。随后，后端调用AI服务层，对用户输入进行语义分析。

AI服务层首先通过Embedding模型完成文本向量化，并在向量数据库中检索相关知识内容。同时，Agent模块根据任务需求决定是否调用外部工具或执行额外操作。系统将检索结果与用户输入共同构造Prompt，并提交给大语言模型生成最终结果。

最终生成结果返回后端服务层，并由前端界面进行展示。

2.3 技术栈架构设计

模块	技术
前端	Vue
后端	FastAPI
AI层	LangChain/原生pipeline
数据库	MySQL
向量库	ChromaDB

3 功能模块详细设计

3.1 用户和前端模块

3.1.1 功能概述

本模块面向普通用户，提供微信小程序端的主要交互界面，承载情绪日记记录、AI角色对话、记忆图谱查看等核心功能的用户入口。

3.1.2 功能列表

功能	描述
微信登录	通过微信授权获取用户身份，完成静默登录
情绪日记	支持文字输入和语音录制两种方式记录日记
AI角色对话	展示预设角色列表，支持与角色进行文本对话
角色创建	用户可上传聊天记录或文本片段，创建自定义AI角色
记忆图谱	以可视化图表展示用户情绪变化趋势和关键事件
个人中心	用户信息查看与修改、历史记录管理

3.1.3 前后端交互说明

前端小程序通过 HTTPS 协议调用后端 RESTful API。所有请求均携带 JWT Token 进行身份认证。核心交互包括：

- **创建日记**：前端提交文本或语音文件 → 后端返回任务状态 → 前端轮询获取分析结果
- **角色对话**：前端发送消息 → 后端组装上下文并调用LLM → 流式或一次性返回回复
- **图谱数据**：前端请求时间范围 → 后端查询聚合数据 → 前端渲染情绪曲线

3.1.4 技术实现要点

- 微信小程序原生开发，使用 TDesign / Vant Weapp 组件库
- 语音录入使用微信录音 API，上传至后端进行 ASR 转写
- 图表可视化使用 ECharts 小程序版渲染情绪曲线和时间轴
- 聊天界面支持打字机效果，提升对话沉浸感

3.2 后台管理模块

3.2.1 功能概述

本模块面向系统管理员，提供 Web 端管理后台，用于系统运维、用户管理及数据监控。

3.2.2 功能列表

功能	描述
用户管理	查看用户列表，管理用户状态（启用/禁用），查看用户活跃情况
日志管理	查看系统操作日志、API 调用日志、异常日志，支持按时间筛选
权限管理	管理员账号管理，角色权限分配
预设角色管理	管理系统预设的 AI 角色（新增、编辑、下架），维护角色画像和示例对话
内容安全审核	查看风险预警记录，处理被标记为高危的会话内容
数据统计	系统使用数据看板（日活用户、日记数量、对话次数、情绪分布等）

3.2.3 前后端交互说明

管理员通过 Web 端登录，所有管理接口需验证管理员权限。核心交互包括：

- **用户管理**：调用用户列表接口，支持分页、搜索、状态筛选
- **日志查询**：调用日志接口，支持时间范围和类型筛选
- **预设角色管理**：调用角色管理接口，创建或更新系统预设角色
- **数据看板**：调用统计聚合接口，获取实时数据展示

3.2.4 技术实现要点

- 后台前端可用 Vue 或 React 构建，与小程序共用同一后端 API
- 管理接口独立权限校验，使用角色-权限模型
- 日志数据存储于 MongoDB，支持大量写入和灵活查询
- 数据看板通过定时任务聚合指标数据，提升查询性能

3.3 AI 智能模块

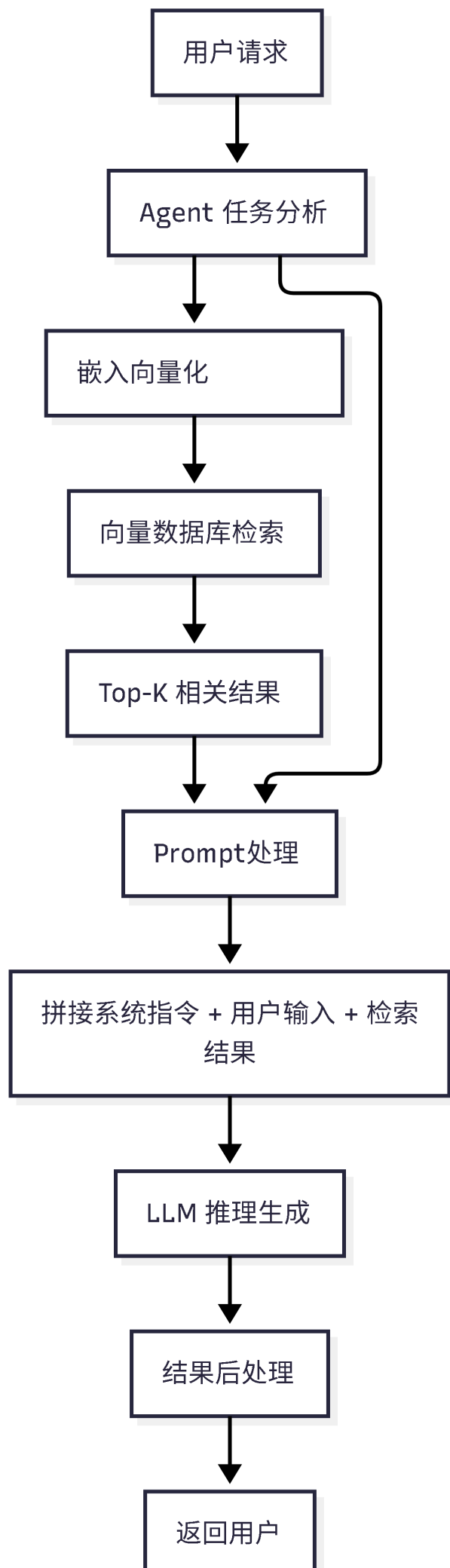
3.3.1 功能概述

AI 智能模块是本系统的技术核心，负责实现情感计算、知识检索、对话生成等智能能力。该模块位于后端服务层与 AI 模型之间的编排层，是连接业务逻辑与大语言模型的桥梁。

3.3.2 模块组成

子模块	职责
Prompt 处理	构造规范化的提示词模板，管理系统指令和角色设定
Embedding	将文本转换为高维向量，支撑语义检索
检索 (RAG)	基于向量相似度检索相关知识内容，增强生成质量
Agent	任务分析与决策调度，协调各子模块协作
生成回答	调用大语言模型生成最终回复

3.3.3 模块间协作关系

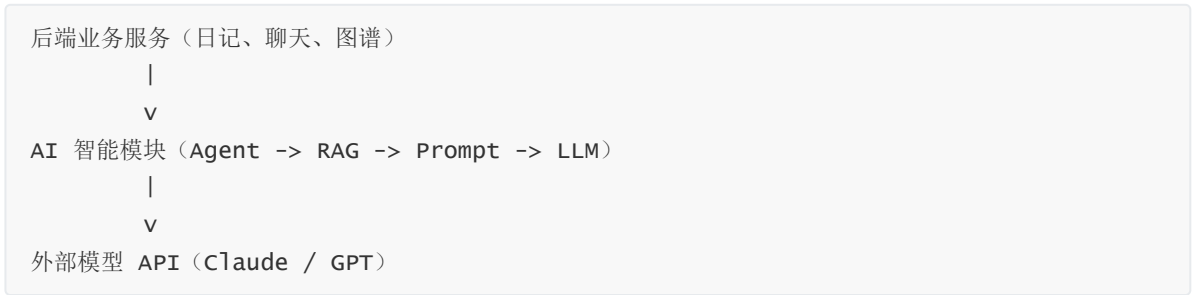


模块间协作关系说明

AI 智能模块各子模块按以下流程协作：

1. **Agent** 接收用户请求，进行任务分析，决定需要调用的子模块。
2. 根据任务类型，Agent 调度 **Embedding** 模块对用户输入或检索内容进行向量化。
3. **检索模块** 在向量数据库中执行语义相似度检索，获取 Top-K 相关结果。
4. **Prompt 处理模块** 将用户输入、检索结果、系统指令拼接为标准化 Prompt。
5. **生成模块** 将 Prompt 提交给大语言模型，获得生成结果。

3.3.4 在系统中的位置



详细设计请参见第 5 章“AI 核心模块设计”。

3.4 数据管理模块

3.4.1 功能概述

数据管理模块负责系统中所有数据的存储组织、访问控制和生命周期管理。它是系统稳定运行的数据基础。

3.4.2 功能列表

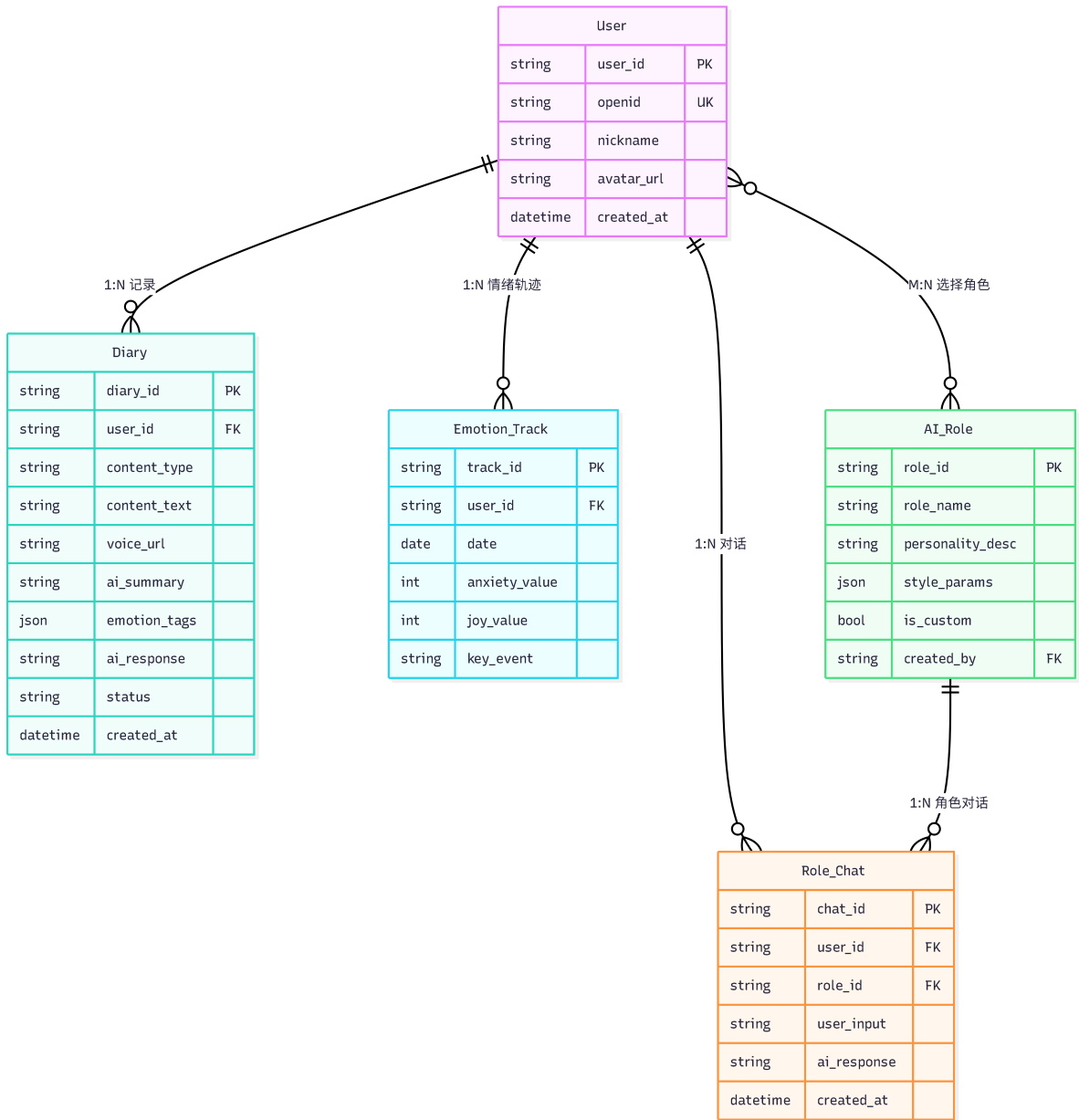
功能	描述
数据上传	支持用户上传文字、语音、文本文件等多种格式数据
数据存储	根据数据类型选用合适的存储引擎（关系型、文档型、向量型、对象存储）
数据查询	提供高效的数据检索能力，支持精确查询和模糊搜索
数据聚合	对情绪数据进行按日/周/月聚合，支撑图谱可视化
数据安全	用户敏感数据加密存储，访问鉴权，数据脱敏

3.4.3 数据分类与存储策略

数据类型	存储引擎	说明
用户信息	MySQL	结构化强关联数据
日记与对话	MongoDB	长文本、非结构化内容
角色画像	MongoDB	灵活的角色属性结构

数据类型	存储引擎	说明
向量数据	pgvector / Milvus	角色语料和对话片段的语义向量
语音文件	对象存储 (OSS/COS)	大文件存储，数据库仅保留访问URL
缓存与会话	Redis	热点数据缓存、会话上下文、任务队列

3.4.4 数据库ER关系



数据库ER关系说明

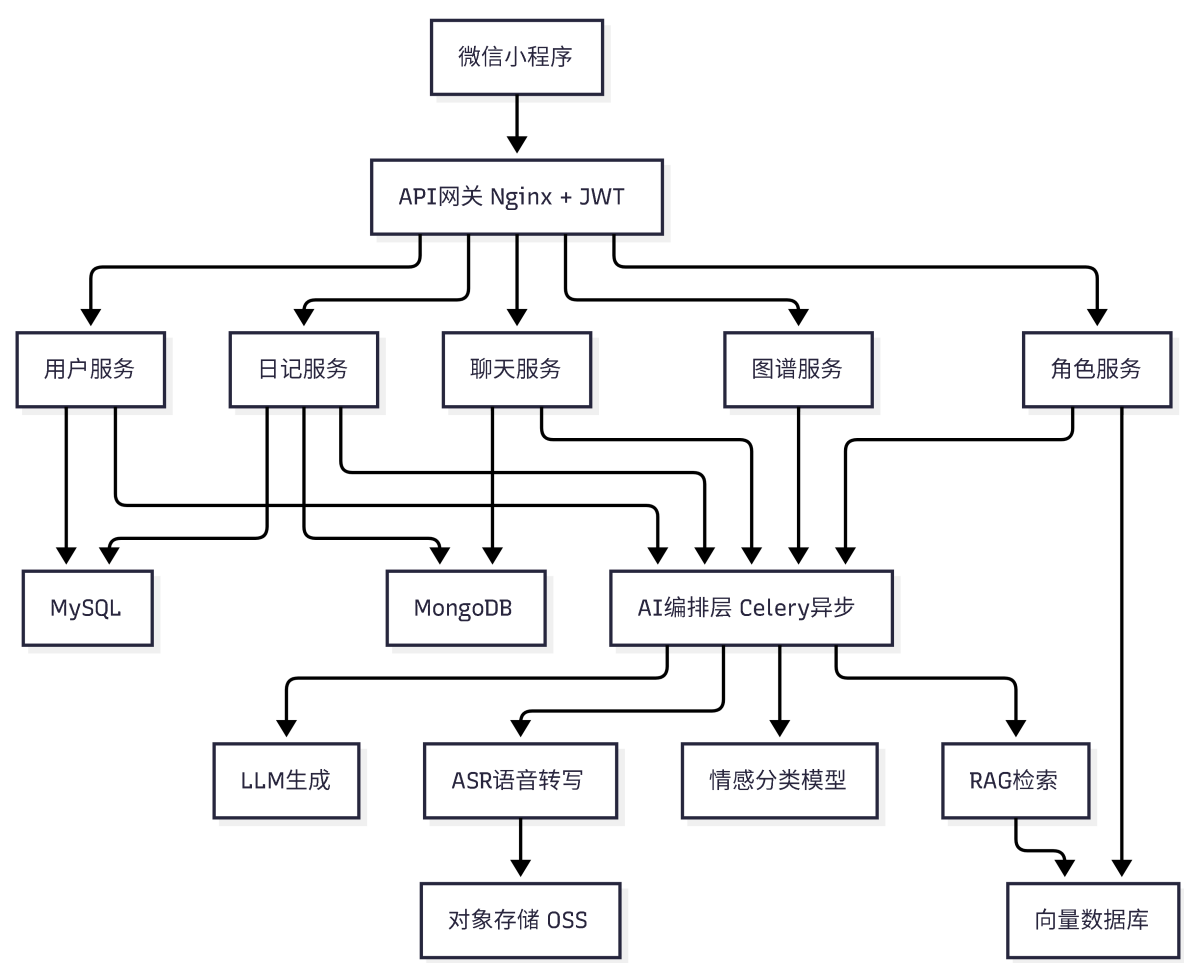
核心实体关系如下：

- **User** 与 **Diary**：一对多（一个用户可有多条日记）
- **User** 与 **AI_Role**：多对多（用户可选择多个角色，角色可被多个用户使用）
- **User** 与 **Role_Chat**：一对多
- **AI_Role** 与 **Role_Chat**：一对多
- **User** 与 **Emotion_Track**：一对多

详细设计请参见配套的“数据库设计说明书”。

4 后端系统设计

4.1 后端架构概览



后端架构分层说明

本项目后端采用**分层、服务化的架构风格**，核心是基于 **Python + FastAPI** 构建的 API 服务层。它作为系统的中枢，向下连接数据存储，向上为前端提供业务接口，内部则与 AI 服务层协同，执行智能任务。

1. 接入层 (Gateway)

- **职责**：统一接收前端请求，负责身份认证、权限校验、请求限流和日志记录
- **实现**：使用 **Nginx** 作为反向代理，结合 **JWT** 实现无状态认证

2. 业务服务层 (Business Services)

- **职责**：实现核心业务逻辑，每个核心功能对应独立服务
- **服务划分**：
 - 用户服务：登录、注册、用户信息管理
 - 日记服务：情绪日记的创建、查询，调度AI分析流程
 - 角色服务：管理预设角色和用户自定义角色
 - 聊天服务：处理角色对话，维护会话状态
 - 图谱服务：查询聚合情绪数据，为可视化提供数据支撑

3. AI 编排层 (AI Orchestration)

- **职责**：将AI处理流程抽象为可复用的任务，协调调用底层模型和工具
- **核心组件**：

- AIService：与 LLM 交互的核心服务
- EmotionAnalyzer：调用情感分类模型
- RAGRetriever：管理向量数据库检索
- TaskManager (Celery)：管理异步任务，处理耗时操作

4. 数据与模型层 (Data & Model)

- 职责：提供持久化存储和外部模型访问
- 组成：MySQL、MongoDB、Redis、向量数据库、对象存储、大模型 API

4.2 技术栈选型

组件	技术选型	选型理由
Web 框架	Python + FastAPI	高性能、异步、原生支持 API 文档自动生成 (Swagger UI)，适合 AI 应用接口服务
API 网关	Nginx	高性能反向代理，处理负载均衡和静态资源
任务队列	Celery + Redis	处理 AI 对话、角色蒸馏等耗时任务，防止阻塞主线程
关系型存储	MySQL	存储用户、角色、日记索引等强关联结构化数据
文档存储	MongoDB	存储非结构化长文本 (日记内容、对话历史)，结构灵活便于扩展
向量存储	pgvector / Milvus	存储角色语料和对话片段的 Embedding 向量，支撑语义检索
缓存	Redis	缓存热点数据、管理会话上下文、作为 Celery 消息代理
AI 模型 API	Claude / GPT API	核心对话生成和文本理解引擎
情感模型	中文情感分类模型	专用结构化情绪识别，提供稳定可控的情绪标签

4.3 核心数据模型设计

基于需求分析，后端核心数据模型及关系如下：

核心表说明

表名	核心字段	说明
User	user_id (PK), openid (唯一), nickname, avatar_url, created_at	用户基本信息，所有业务数据的外键关联基础

表名	核心字段	说明
Diary	diary_id (PK), user_id (FK), content_type (text/voice), content_text, voice_url, ai_summary, emotion_tags (JSON), ai_response, status (pending/completed), created_at	情绪日记记录，关联 AI 分析结果
AI_Role	role_id (PK), role_name, personality_desc, style_params (JSON), is_custom (布尔), created_by (FK, 自定义角色时关联用户)	AI 角色信息，区分预设和用户自定义
Role_Chat	chat_id (PK), user_id (FK), role_id (FK), user_input, ai_response, created_at	角色对话记录，支撑上下文记忆
Emotion_Track	track_id (PK), user_id (FK), date, anxiety_value (0-100), joy_value (0-100), key_event (文本)	按日聚合的情绪数据，支撑图谱可视化

实体关系

- User 与 Diary: **一对多**（一个用户可有多条日记）
- User 与 AI_Role: **多对多**（用户可选择多个角色，角色可被多个用户使用）
- User 与 Role_Chat: **一对多**
- AI_Role 与 Role_Chat: **一对多**
- User 与 Emotion_Track: **一对多**

4.4 API 接口设计

所有 API 遵循 RESTful 风格，使用 HTTPS 协议，返回统一 JSON 格式。

4.4.1 统一响应格式

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

4.4.2 用户服务接口

API 端点	方法	功能描述	认证
/api/v1/auth/login	POST	微信登录，换取 JWT Token	否
/api/v1/users/me	GET	获取当前用户信息	是
/api/v1/users/me	PUT	更新用户信息	是

4.4.3 日记服务接口

API 端点	方法	功能描述	认证
/api/v1/diaries	POST	创建日记（文本或语音），提交异步分析	是
/api/v1/diaries	GET	获取日记列表，支持分页和情绪标签筛选	是
/api/v1/diaries/{id}	GET	获取日记详情（含 AI 分析结果）	是

4.4.4 角色服务接口

API 端点	方法	功能描述	认证
/api/v1/roles	GET	获取可用角色列表	是
/api/v1/roles	POST	创建自定义角色，启动角色蒸馏任务	是
/api/v1/roles/{id}	GET	获取角色详细信息	是

4.4.5 聊天服务接口

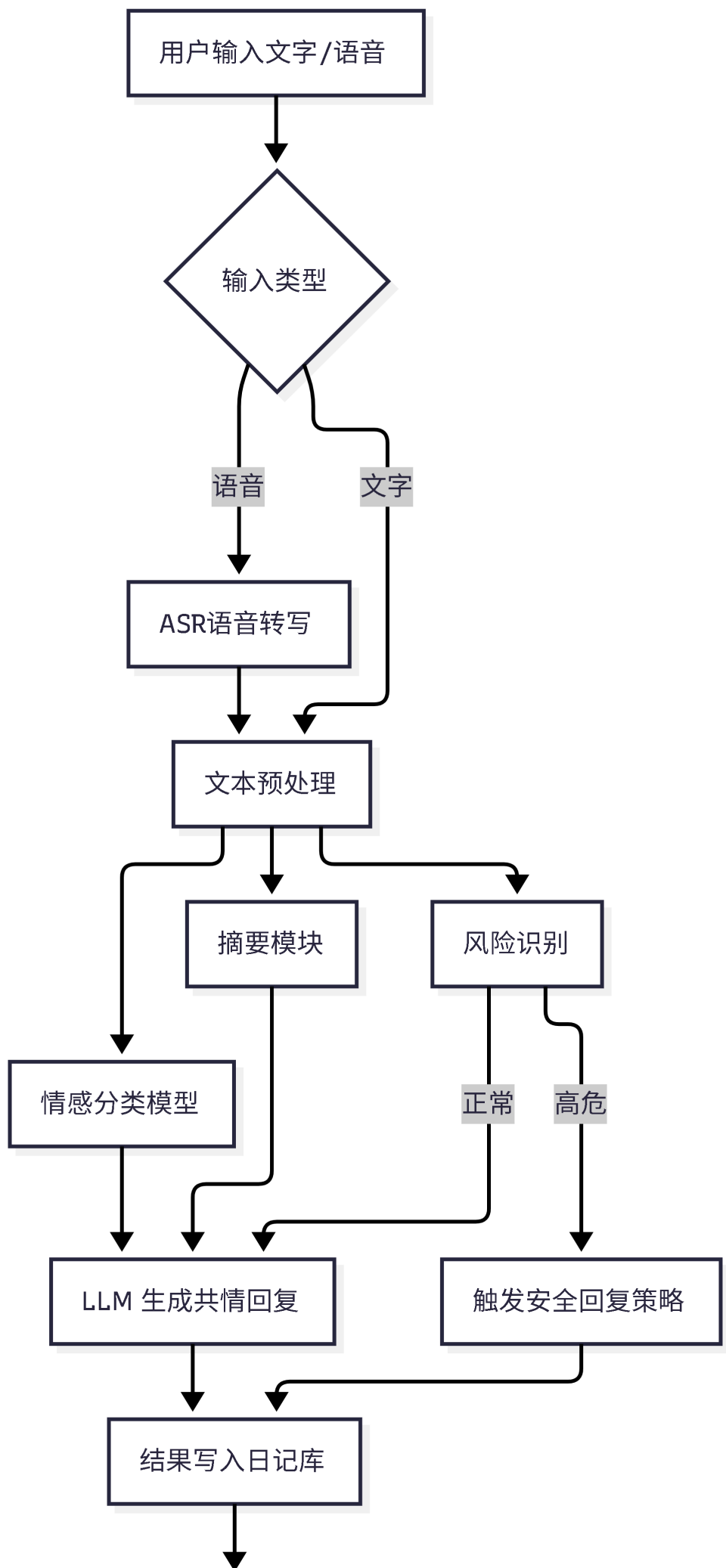
API 端点	方法	功能描述	认证
/api/v1/chat/sessions	POST	创建对话会话	是
/api/v1/chat/sessions/{id}	POST	发送消息，获取 AI 回复	是
/api/v1/chat/sessions/{id}/messages	GET	获取历史消息	是

4.4.6 图谱服务接口

API 端点	方法	功能描述	认证
/api/v1/graphs/emotion	GET	获取情绪图谱数据，支持时间范围和类型筛选	是

4.5 后端核心业务流程设计

4.5.1 情绪日记与 AI 共情流程

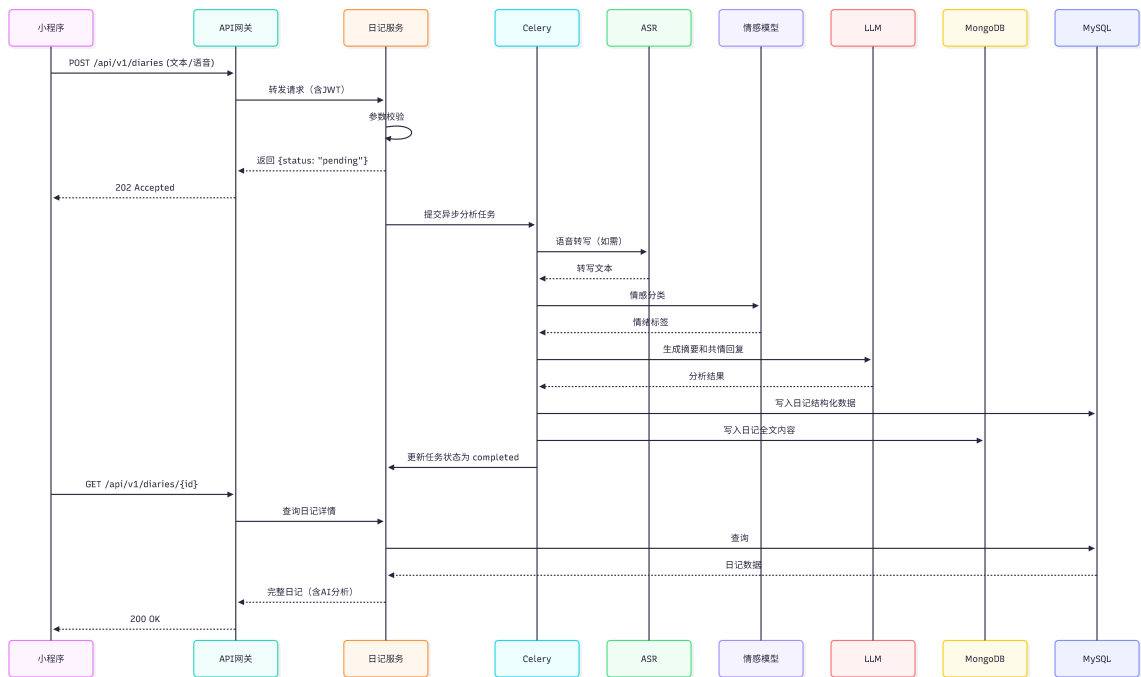


更新情绪时间线

情绪日记与AI共情流程说明

1. 用户输入文字或语音，若为语音则先经过 ASR 转写。
2. 文本预处理后，依次调用情感分类模型、摘要模块、风险识别模块。
3. 根据情绪标签和风险等级，LLM 生成共情式回复；高风险场景触发安全回复策略。
4. 最终将分析结果写入日记库，并更新情绪时间线。

4.5.2 情绪日记异步处理时序

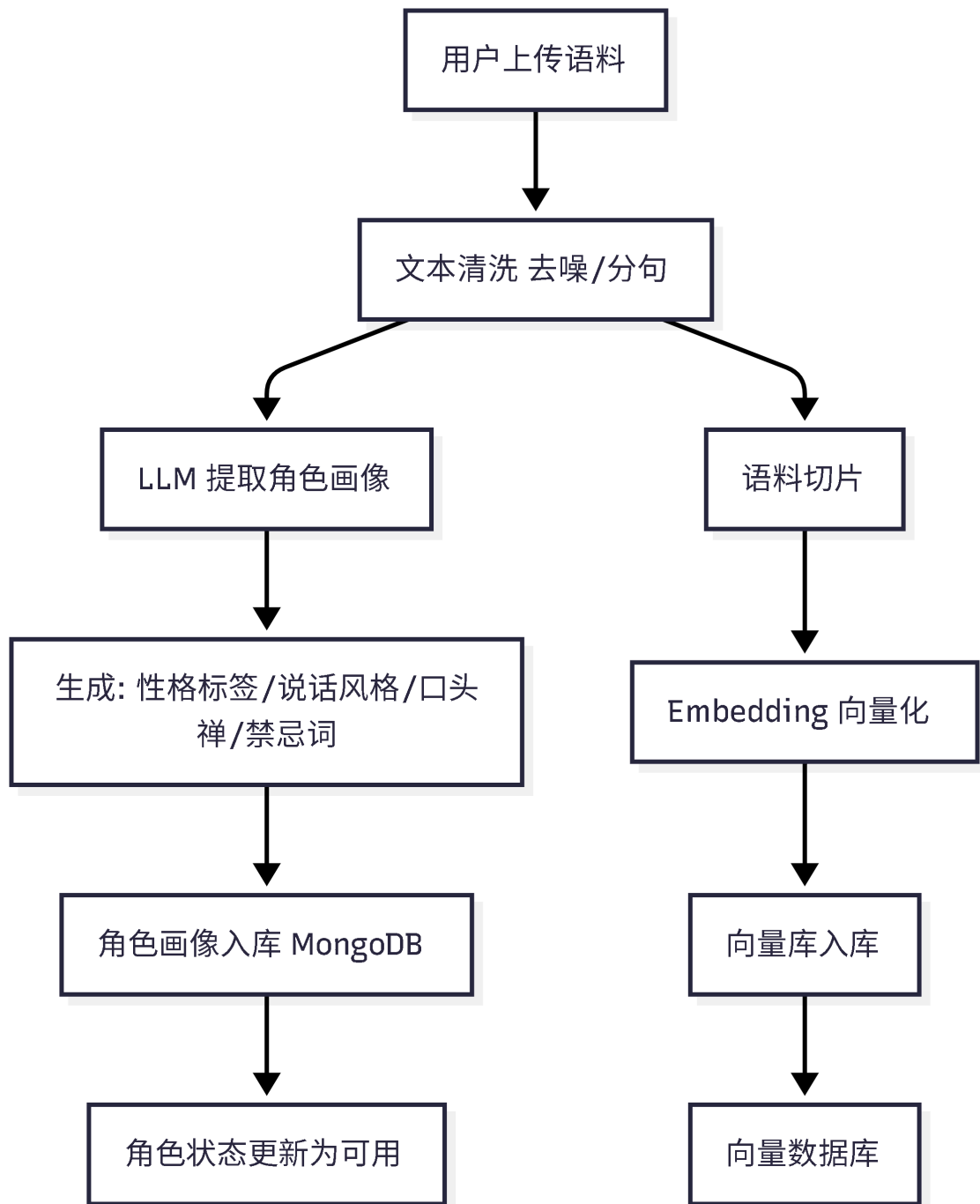


情绪日记异步处理时序说明

前端提交日记后，后端立即返回“pending”状态，由 Celery 异步执行语音转写、情感分析、摘要生成和 LLM 回复。处理完成后前端轮询获取详细结果。

4.5.3 AI 角色蒸馏与对话流程

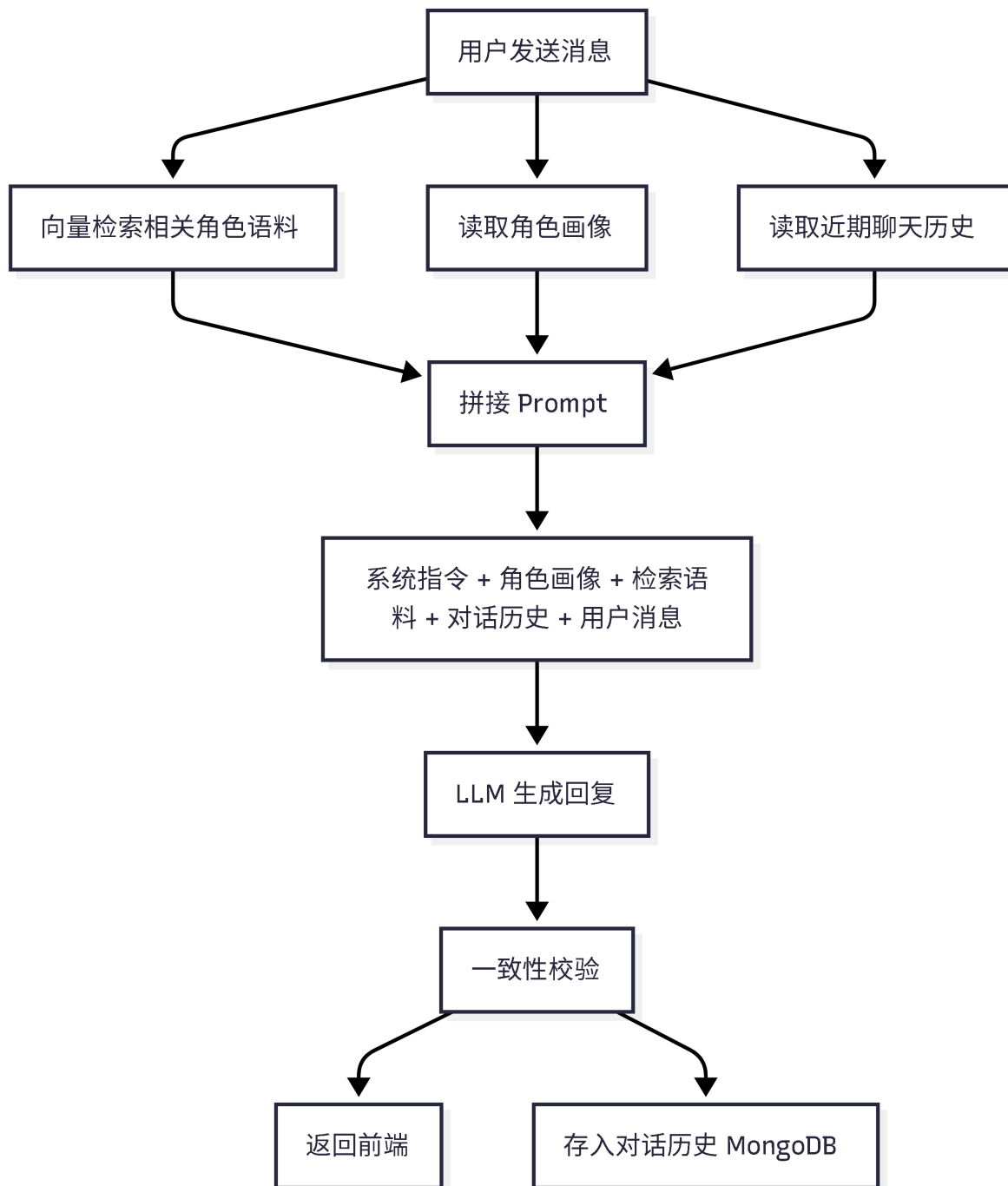
创建角色阶段



创建角色阶段说明

1. 用户上传语料，经文本清洗后，由 LLM 提取角色画像（性格、风格、口头禅等）。
2. 语料切片经 Embedding 向量化后存入向量数据库，角色画像存入 MongoDB。
3. 处理完成后角色状态更新为可用。

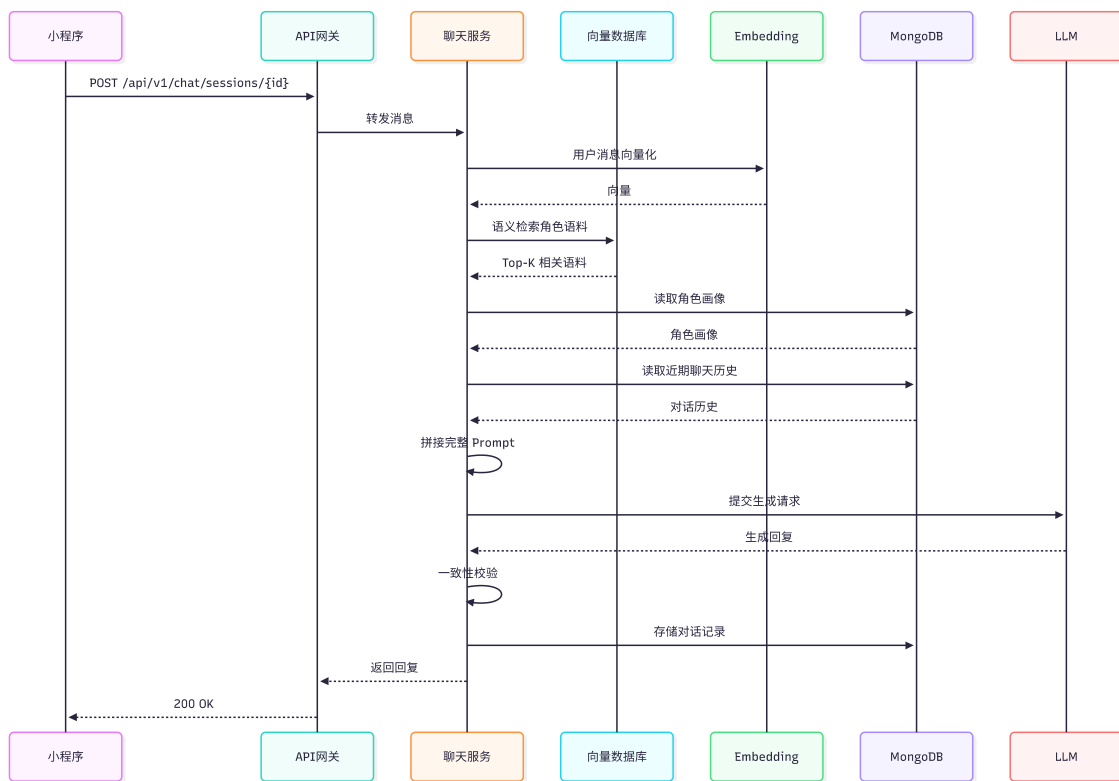
实时对话阶段



实时对话阶段说明

1. 用户发送消息后，系统从向量库检索相关角色语料，同时读取角色画像和近期聊天历史。
2. 将系统指令、角色画像、检索语料、对话历史与当前消息拼接为完整 Prompt。
3. LLM 生成回复，经过一致性校验后返回前端，并存入对话历史。

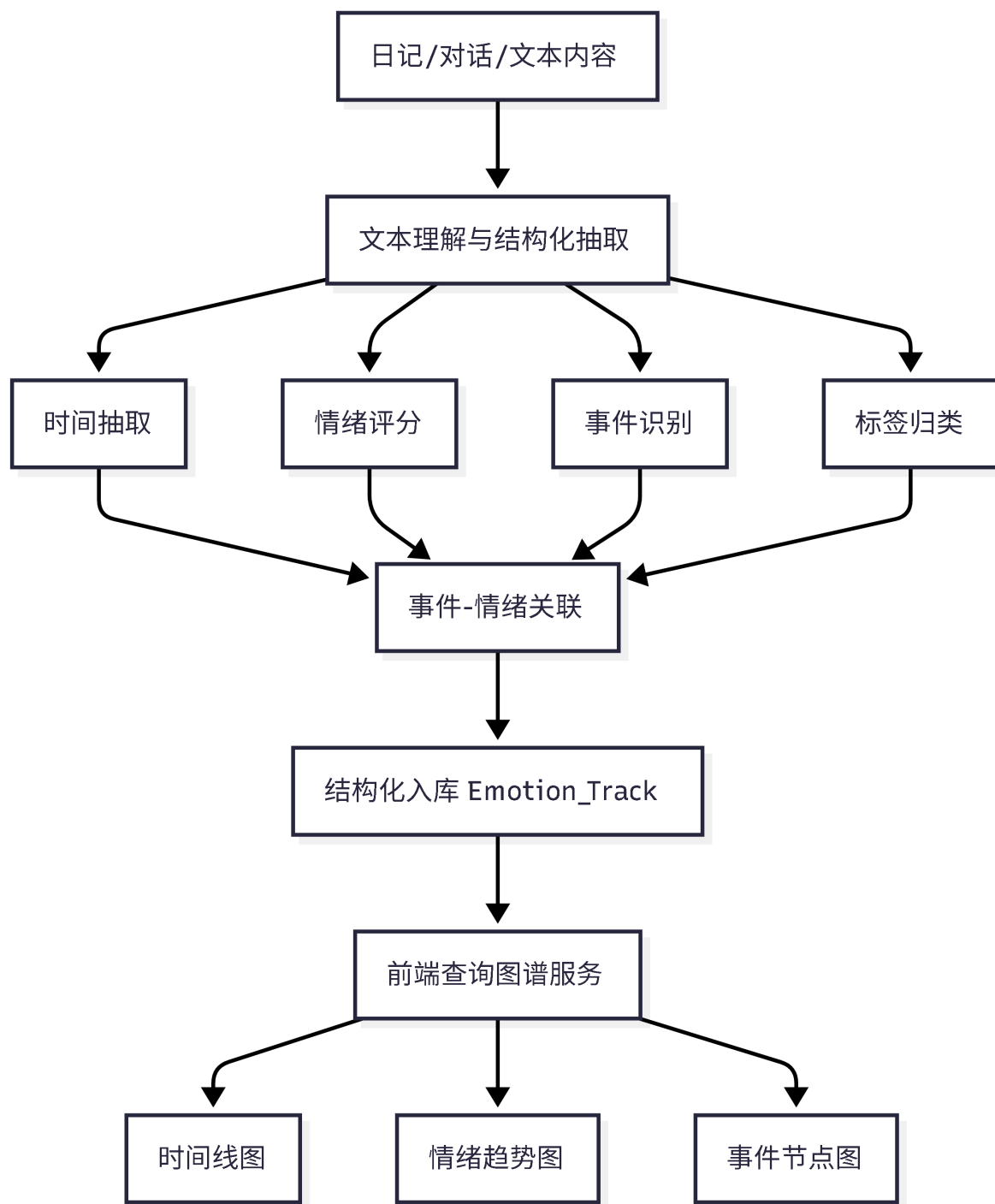
4.5.4 角色对话完整时序



角色对话完整时序说明

聊天服务收到消息后，依次访问 Embedding 模型、向量数据库、MongoDB（画像与历史），组装 Prompt 后调用 LLM，最终返回回复并存储记录。

4.5.5 记忆图谱数据流程



记忆图谱数据流程说明

1. 从日记、对话等历史数据中抽取时间、情绪评分、事件、标签等结构化信息。
2. 事件与情绪关联后写入 MySQL 的 `Emotion_Track` 表。
3. 前端查询图谱服务，渲染时间线、情绪趋势图和事件节点图。

4.6 非功能需求实现

安全性

- **认证**：所有需保护 API 通过 JWT 进行访问控制
- **加密**：敏感数据（日记内容）使用 AES-256 加密存储
- **风控**：日记分析任务强制加入高危关键词检测，命中后触发安全回复策略

性能

- **异步化**：使用 Celery 处理所有耗时任务，避免 API 超时
- **高并发**：Nginx + FastAPI (async) 提供异步非阻塞 IO 能力
- **水平扩展**：可独立增加 Celery Worker 或 API 容器节点提升吞吐量

可用性

- API 接口遵循 RESTful 规范，返回格式统一
- 错误响应包含明确的错误码和描述信息
- 关键操作有日志记录，便于排查问题

可扩展性

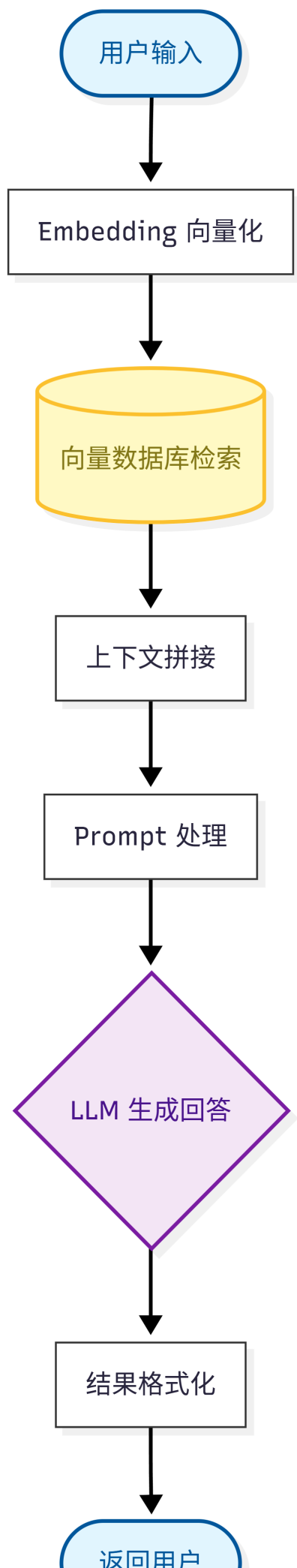
- 业务服务按功能拆分，新增功能只需增加新服务而非修改现有服务
- AI 模型可灵活切换（Claude / GPT 或其他开源模型）
- 数据库支持从 pgvector 迁移到 Milvus，无需修改业务层代码

非功能需求指标

指标	目标值
API 响应时间	≤ 3秒
并发用户数	≥ 100
系统可用性	≥ 99.5%
数据加密	AES-256

5 AI核心模块设计

5.1 AI模块总体架构



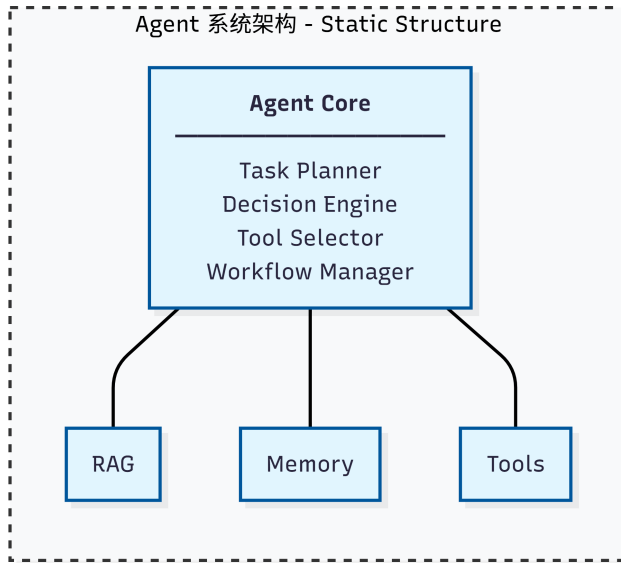
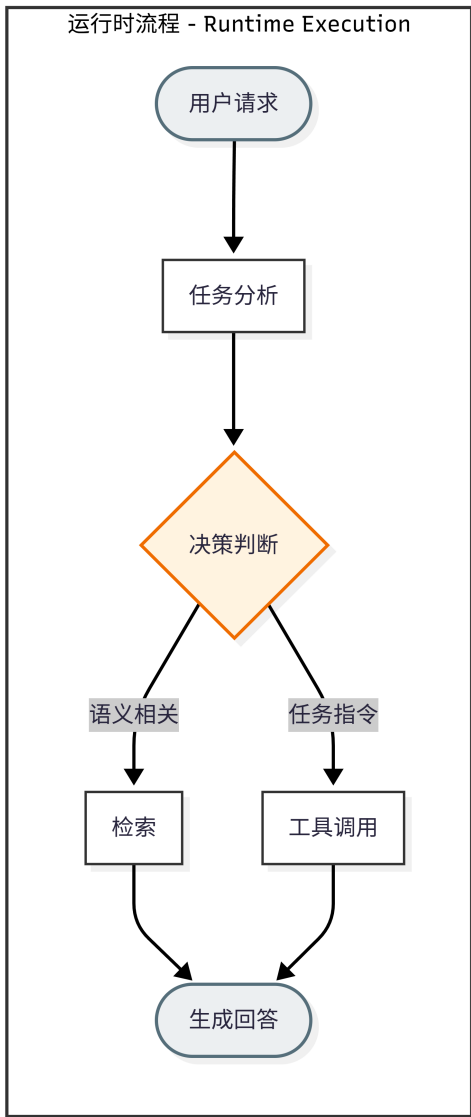
AI管线运行流程说明

系统采用基于RAG（Retrieval-Augmented Generation）的AI处理流程。用户输入首先经过Prompt预处理模块进行规范化处理，随后通过Embedding模型转换为向量表示。

系统利用向量数据库完成语义相似度检索，获取与用户问题相关的知识内容。之后，系统将用户输入与检索结果共同拼接形成增强Prompt，并提交给大语言模型进行推理生成。

最后，系统对生成结果进行格式化处理并返回给用户。

5.2 Agent模块设计

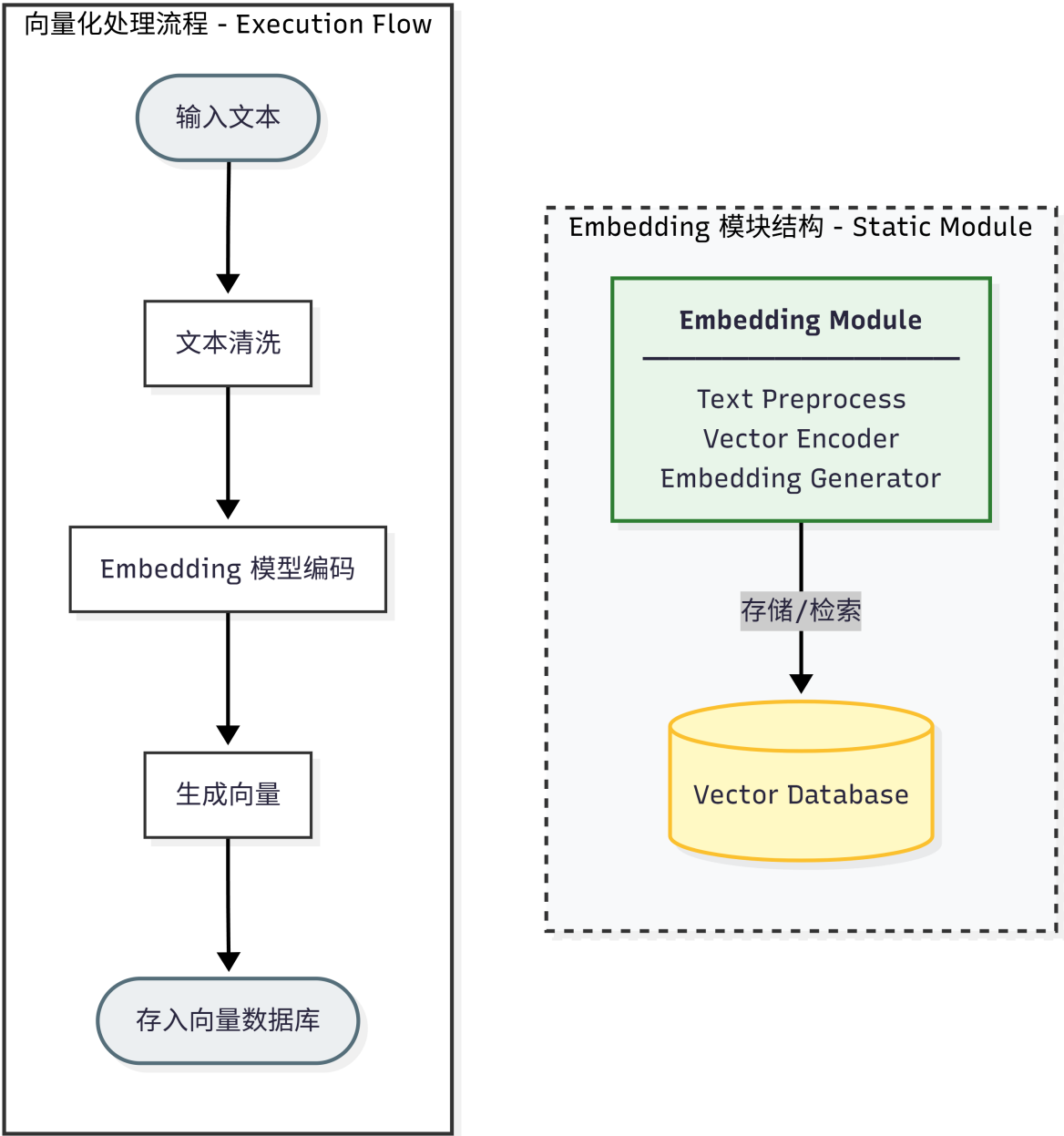


Agent模块设计说明

Agent模块是AI系统的核心调度模块，负责对用户请求进行任务分析与决策控制。系统首先对用户输入进行语义分析，并根据任务类型判断是否需要执行知识检索、工具调用或上下文记忆读取。

Agent模块通过统一 workflow 管理各子模块之间的协作，从而实现复杂任务的智能化处理。

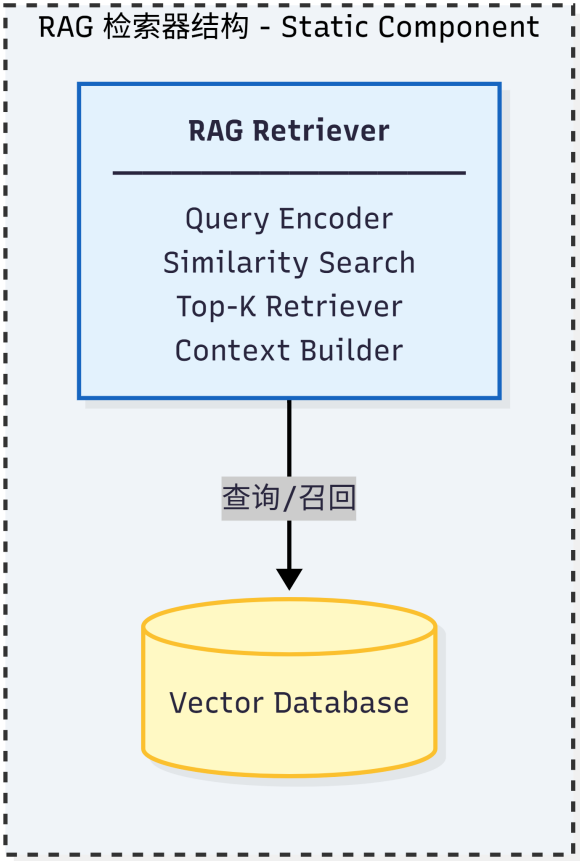
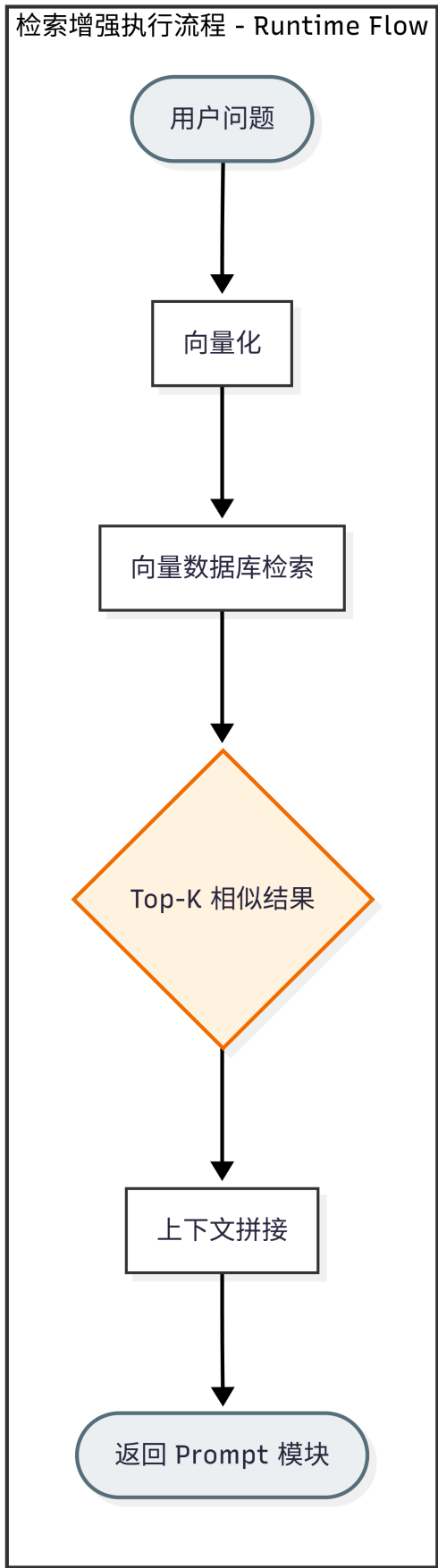
5.3 Embedding模块设计



Embedding模块设计说明

Embedding模块负责将用户输入文本转换为高维向量表示，从而实现语义级别的文本检索与匹配。系统首先对文本进行预处理，随后通过Embedding模型生成向量表示，并存储至向量数据库中供后续检索使用。

5.4 RAG检索模块设计

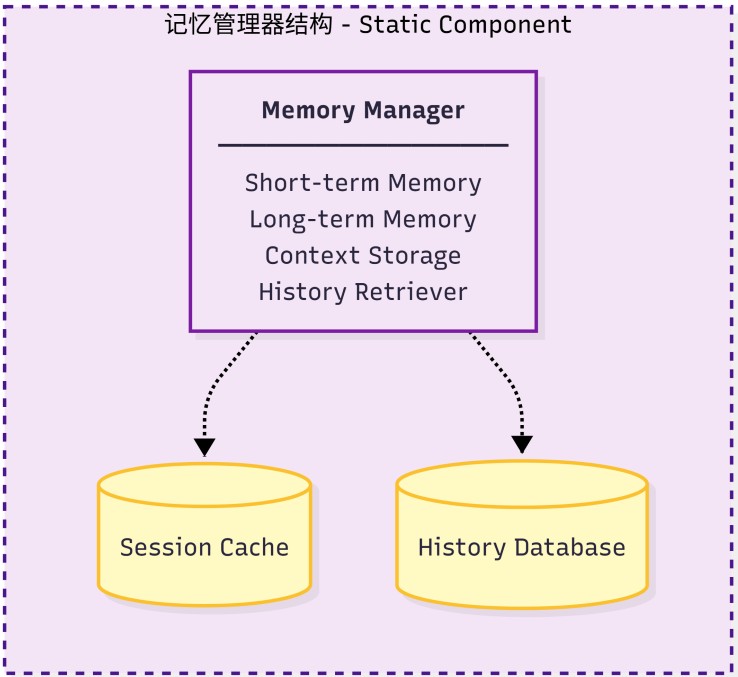
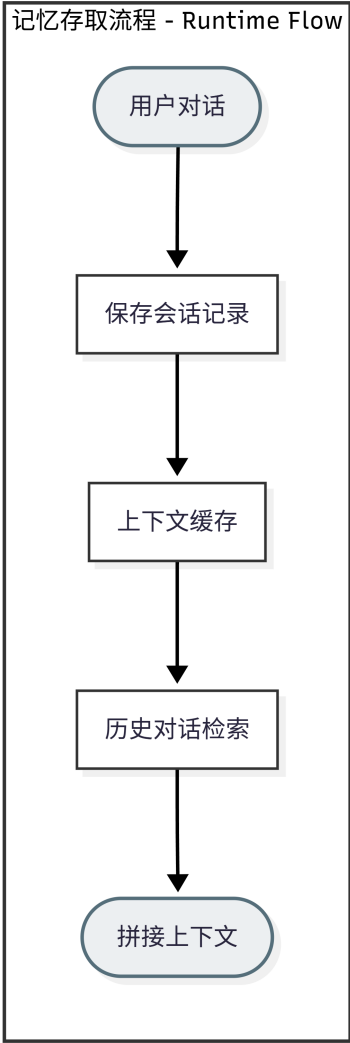


RAG模块设计说明

RAG (Retrieval-Augmented Generation) 模块负责实现基于知识库的语义检索增强。系统首先对用户问题进行向量化处理，并在向量数据库中执行相似度检索，获取与问题最相关的知识内容。

随后，系统将检索结果与用户输入共同拼接形成增强上下文，从而提高大语言模型生成结果的准确性与专业性。

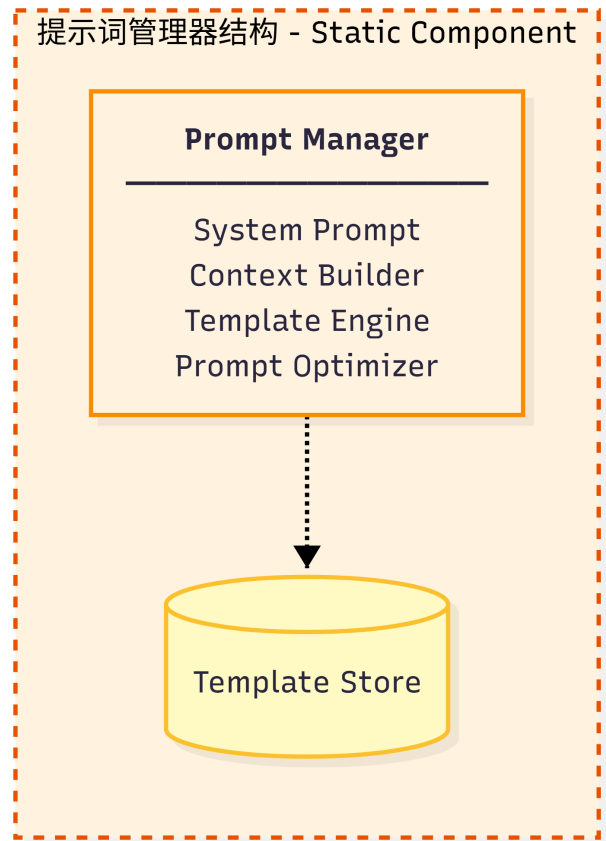
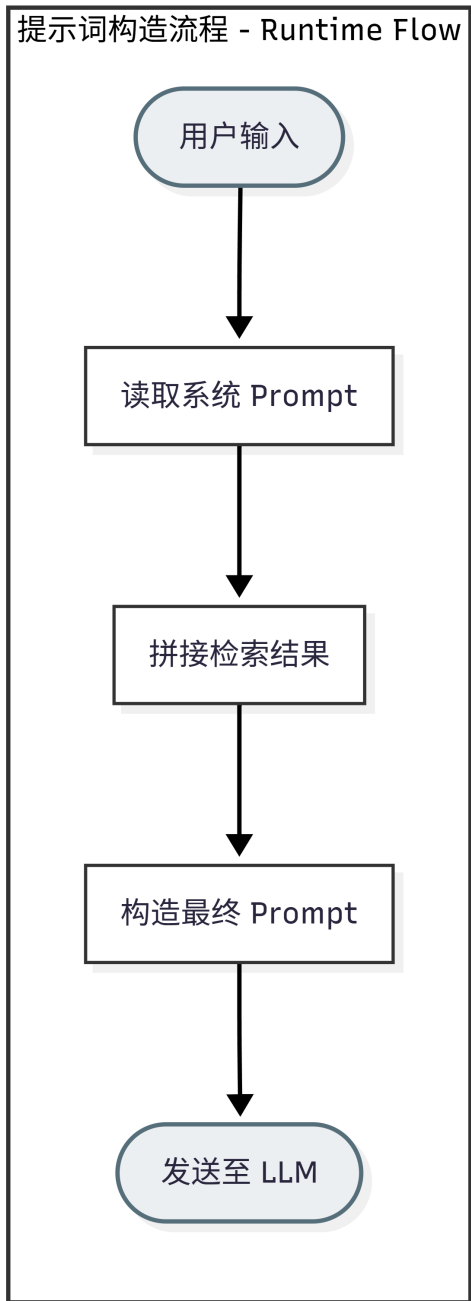
5.5 Memory模块设计



Memory模块设计说明

Memory模块负责系统历史上下文管理，用于维护用户长期与短期对话状态。系统通过缓存历史会话信息，实现上下文连续性，从而提高多轮对话的逻辑一致性与用户体验。

5.6 Prompt工程设计



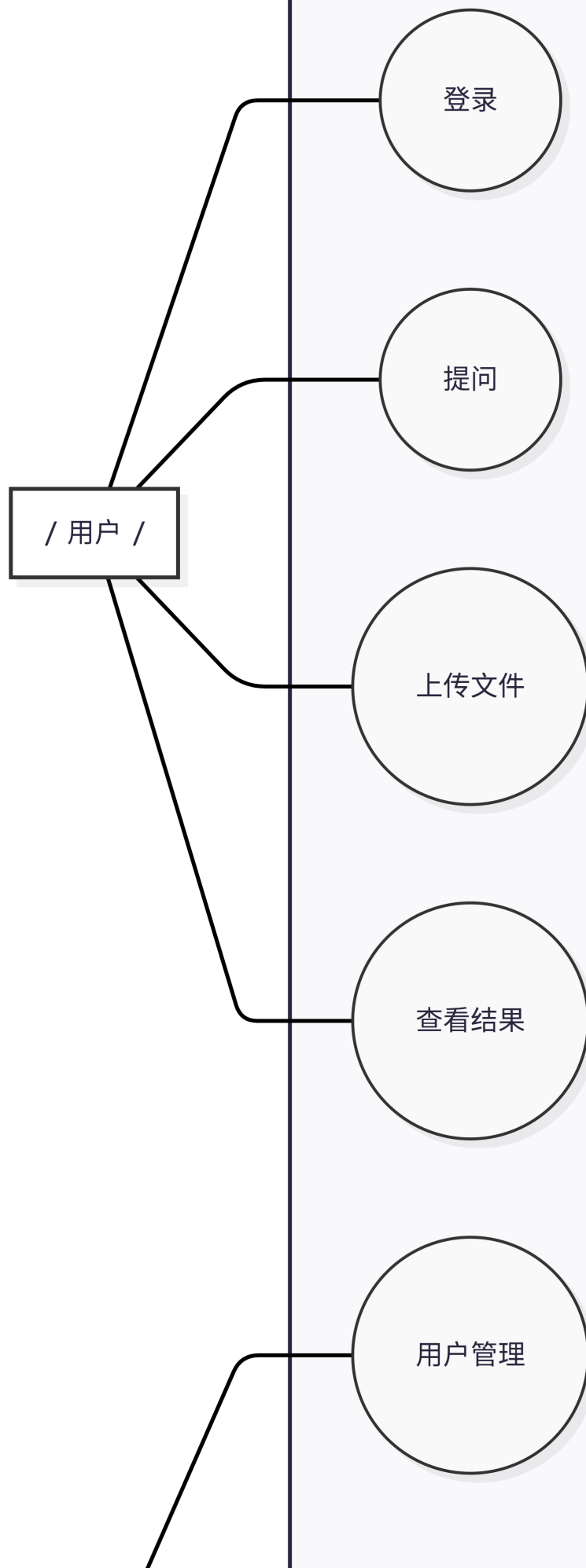
Prompt模块设计说明

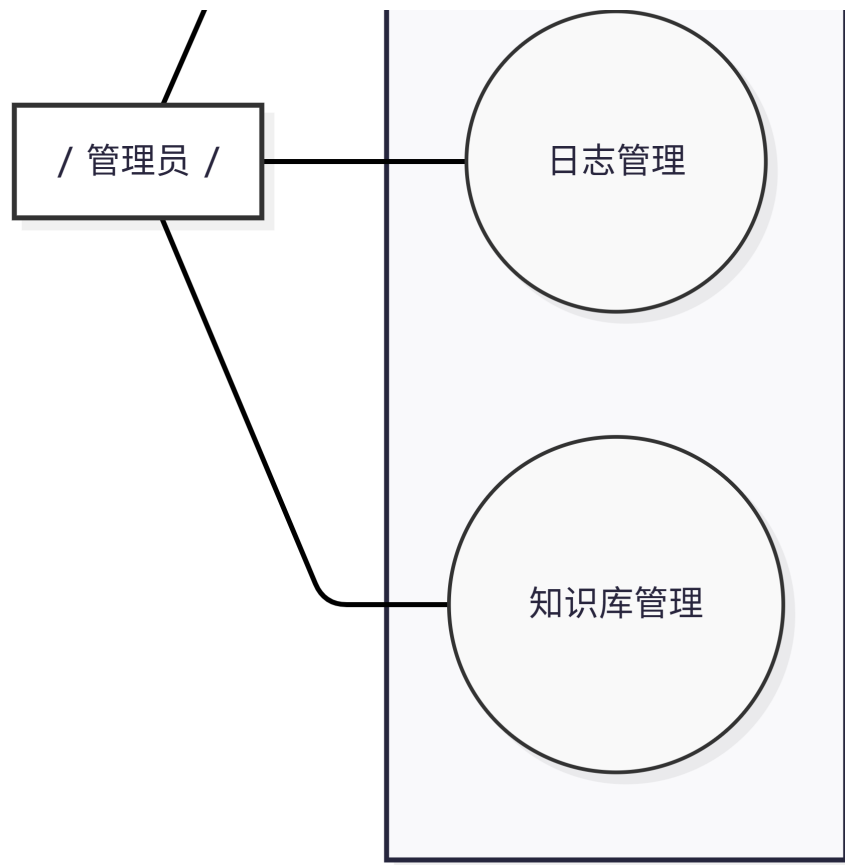
Prompt模块负责构造最终提交给大语言模型的输入内容。系统将用户问题、系统角色设定以及知识检索结果进行统一拼接，并通过模板化方式生成标准Prompt，从而提升模型回答质量与稳定性。

6 UML设计

6.1 用例图

用例图- 功能边界定义

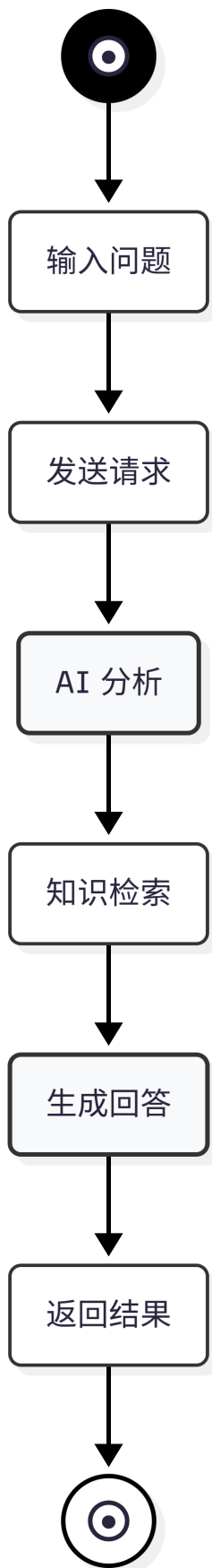




用例图说明

系统主要包含普通用户与管理员两类角色。普通用户可完成登录、智能问答、文件上传及结果查看等功能；管理员则负责系统维护、用户管理以及知识库管理等后台功能。

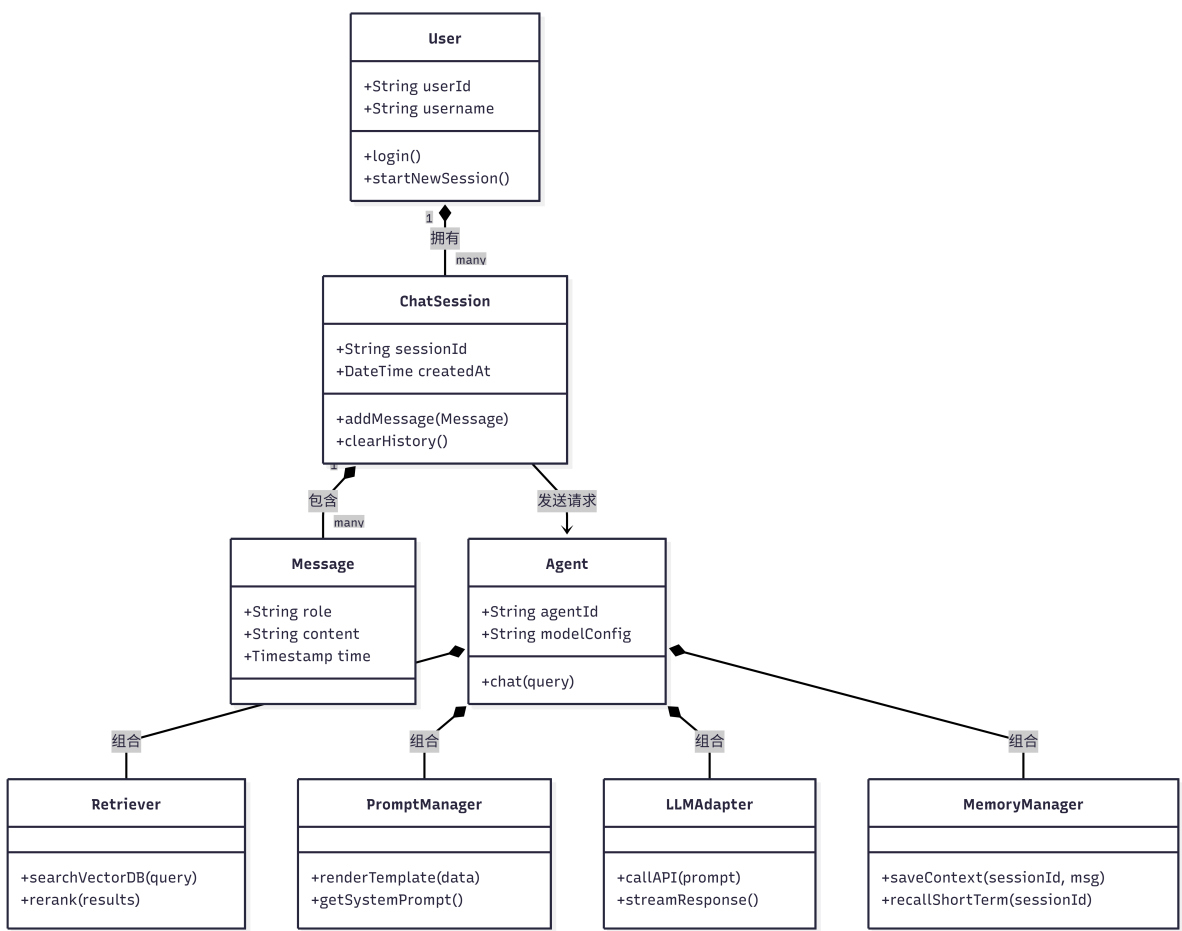
6.2 活动图



活动图说明

当用户提交问题后，系统首先接收用户请求，并调用AI服务层进行语义分析。随后系统完成知识检索与上下文增强，并基于大语言模型生成最终回答，最终将结果返回给用户。

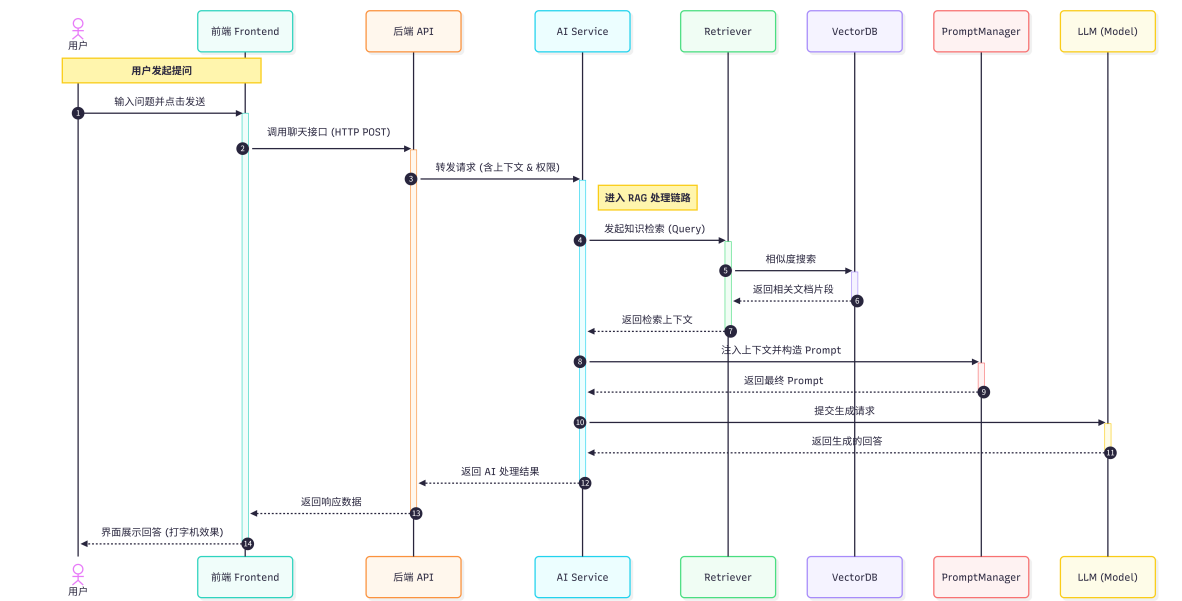
6.3 类图



类图说明

系统主要由用户管理类、会话管理类以及AI服务类构成。其中，Agent类负责整体AI任务调度，并调用Retriever、PromptManager以及LLMAdapter等子模块完成知识检索与文本生成。

6.4 时序图



时序图说明

用户提交请求后，前端将请求发送至后端API服务层。后端调用AI服务层进行处理，其中Retriever模块负责知识检索，PromptManager负责上下文构造，LLM模块负责生成最终回答。最终结果通过后端返回前端并展示给用户。

7 接口设计

7.1 前后端接口

API	功能
/chat	对话
/login	登录

7.2 AI服务接口

接口	功能
embed()	向量化
retrieve()	检索
generate()	生成